# Cadenas de texto

Las cadenas de texto o Strings son una secuencia de caracteres entre comillas, ya sean simples o dobles.

```
exto1 = "Hola mundo"
texto2 = 'Python es genial'
texto3 = """Cadena
en varias líneas"""
```

## Indexación y slicing

Puedes acceder a los carácteres de la cadena de texto a través de su índice.

> *Recuerda, como todos los lenguajes de programación de confianza, los índices empiezan en 0.*

```
texto = "python"
print(texto[0]) # imprime p
print(texto[-1]) # imprimir n (-1 accede al último índice de la cadena, la última posición)
```

### Slicing o subcadenas

En python se pueden coger framentos de cadenas de dos maneras, a través de funciones o haciendo uso de `[inicio:fin:salto]`, el funcionamiento se parece al range pero en este caso se puede hacer sobre colecciones indexadas (en este caso se hace sobre cadenas de texto, pero veremos que se puede usar sobre listas).

```
texto = "python"
print(texto[0:3]) # imprime pyt
print(texto[:4]) # imprime pyth (desde el inicio hasta 3)
print(texto[2:]) # imprime thon (desde la posición 2 hasta el final)
print(texto[::-1]) # imprime nohtyp (no establecemos posición de inicio ni de final, pero especificamos que el salto es en sentido contrario)
```

## Funciones y otras utilidades

### Recorrer cadenas con bucles

Se puede utilizar un bucle `for` para recorrer cada elemento que compone la cadena de texto, este interpretará cada carácter como si fuera un elemento a recorrer.

```
texto = "python"
for letra in texto:
    print(letra)
```

Otra opción (aunque mucho menos recomendada) es hacerlo con un bucle `while`, aunque es algo más farragoso ya que recorremos por índices, no por elementos:

```
texto = "python"
i = 0
```

```
while i < len(texto):
    print(texto[i])
```

### Longitud

```
len("hola") # 4
```

> *`len` cuenta el número de elementos en una colección. A si que no solo nos van a ser útiles contando el número de carácteres en las cadenas de texto*

### Transformación de mayúsculas/minúsculas

```
texto = "Python"
texto.upper() # 'PYTHON'
texto.lower() # 'python'
texto.title() # 'Python'
texto.capitalize()# 'Python'
texto.swapcase() # 'pYTHON'
```

- `upper()` pone la cadena en mayúsculas.
- `lower()` pone la cadena en minúsculas.
- `title()` pone la cadena en formato título, la primera letra de cada palabra en mayúsculas y el resto de cada palabra en minúsculas.
- `capitalize()` pone la primera letra de la cadena en mayúsculas y el resto en minúsclas.
- `swapcase()` cambia las mayúsculas por minúsuculas y viceversa.

### Búsqueda y verificación

```
texto = "python"
texto.startswith("py") # True
texto.endswith("on") # True
texto.find("th") # 2 (posición)
texto.count("o") # 1 (cantidad)
```

- `startswith(cadena)` verifica si una cadena de texto contiene la `cadena` pasada por parámetro.
- `endswith(cadena)` verifica si una cadena de texto contine

### Reemplazo, partición y unión

```
texto.replace("Py", "Ru") # 'Ruthon'
texto.split(" ") # ['Hola', 'mundo']
" ".join(["Hola", "mundo"]) # 'Hola mundo'
```

- `replace(cadena1, cadena2)` reemplaza en la cadena de texto de origen la `cadena1` por la `cadena2` .
- `split(cadena)` devuelve un array creado a partir de la división de la cadena de texto tomando `cadena` como valor por el cual dividir.
- `join(lista)` junta el contenido de la `lista` pasada por parámetro utilizando la cadena de texto como unión, en el ejemplo, se utiliza el carácter espacio `" "`.

### Formateo de cadenas

### Concatenación

```
nombre = "Python"
saludo = "Hola " + nombre
```

### f-strings

```
nombre = "Python"
edad = 34
saludo = f"Hola {nombre}, tienes {edad} años"
```

### .format()

```
nombre = "Python"
edad = 34
saludo = "Hola {}, tienes {} años".format(nombre, edad)
```

## Otras utilidades

### Repetir cadenas

```
print("Hola " * 3)  # Hola Hola Hola
```

### Verificar si contiene una subcadena

```
"Py" in texto # True
"Java" not in texto # True
```

### Eliminar espacios

```
"  Hola  ".strip()  # 'Hola'
"  Hola  ".lstrip() # 'Hola  '
"  Hola  ".rstrip() # '  Hola'
```

### Reemplazar contenido con índices

```
texto = "Python"
texto[0] = "C" # Error
texto = "C" + texto[1:]  # 'Cython'
```